SCALE FOR PROJECT AGENT SMITH (/PROJECTS/AGENT-SMITH)

You should evaluate 2 students in this team

★

Git repository

`git@vogsphere.42porto.com:vogsphere/intra-uuid-79a245bd-bbaa-41f8-8e`

Introduction

- Remain polite, respectful and constructive during the evaluation.
- This project is technically demanding and evaluates architecture, security, metrics tracking and AI agent design.
- Carefully verify compliance with the full technical specification.
- Reference exam scripts are provided in the `exams/` directory: `exam_sandbox.sh`, `exam_mbpp.sh`, `exam_swebench.sh`, `exam_anticheat.sh`
- See `exams/exam_scripts_instructions.md` for detailed corrector guidance.
- The correction is designed so long-running tasks (SWE-bench ~45 min, MBPP ~5 min) run in the background while you inspect code.
- Expected total correction time: approximately 60-90 minutes.

Guidelines

- Only grade the work present in the student's Git repository.
- Clone the repository in an empty directory.
- Run `uv sync` before any test.
- The program must not crash unexpectedly.
- The student solution must NEVER import moulinette packages.
- If a crash occurs during execution, the final grade is 0.
- No modification of student code is allowed.
- Docker containers must be cleaned even after forced termination.
- Check iteration limits, token limits and timeout limits strictly.
- Use flags if cheating or forbidden packages are detected.
- Verify the program loads API keys from environment variables / .env file.
- Verify student code does not import agent orchestration libraries (llama-index, smolagents, langgraph, crewai, autogen).
- Verify the agent solves tasks through reasoning and code exploration. Fetching solutions from pull requests, issues, or external sources is strictly forbidden and results in a grade of 0.

Attachments

exams.zip (https://cdn.intra.42.fr/document/document/48844/exams.zip)

subject.pdf (https://cdn.intra.42.fr/pdf/pdf/204085/en.subject.pdf)

moulinette.zip (https://cdn.intra.42.fr/document/document/48845/moulinette.zip)

Mandatory Part

API Key Setup & Free Tier Verification

Do this FIRST so that API keys are ready for the background runs.

Step 1 - Free tier verification:

- Ask the student to show their LLM provider(s) account(s) dashboard (OpenRouter, Groq, or similar)
- Verify the account(s) are on a free tier (no paid plan)
- Ask the student to generate NEW API key(s) live during correction
- The student should be able to explain which provider(s) and model(s) they use and why

Step 2 - Environment variable verification:

- Place the new API key(s) in the .env file
- Verify the agent does NOT have hardcoded API keys in the code

Search student code for hardcoded keys:
grep -rn "sk-" student/ --include="*.py"*
grep -rn "api_key\s=" student/ --include=".py"*

☑ Yes ☒ No

Launch SWE-bench & MBPP (Background)

Launch both exam scripts NOW so they run in the background while
you inspect the codebase. SWE-bench takes ~45 minutes, MBPP ~5 minutes.

In two separate terminals, run:

Terminal 1 — MBPP (fast, ~5 min)
./exams/exam_mbpp.sh --student-path ./student --moulinette-path ./moulinette --env-file .env

Terminal 2 — SWE-bench (slow, ~45 min)
./exams/exam_swebench.sh --student-path ./student --moulinette-path ./moulinette --env-file .env

Verify:

- Both scripts start without immediate errors
- The .env file is loaded correctly (no API key errors)
- SWE-bench begins pulling Docker images

Continue with the remaining questions while these run. MBPP results
are checked in Q8, SWE-bench results in Q14.

This question passes if both scripts launch successfully.

☑ Yes ☒ No

CLI Interface Compliance

Verify required commands:

- uv run python -m agent_mbpp --help
- uv run python -m agent_swebench --help
- uv run sandbox
- uv run sandbox <config.json>
- uv run sandbox --mcp-stdio "python mcp_tools_mbpp.py"
- uv run sandbox --mcp-stdio "python mcp_tools_swebench.py"
- uv run sandbox --mcp-server <URL>

Ensure commands run correctly and do not crash.

Verify the program loads API keys from environment variables
(e.g., from a .env file).

Reference: exams/exam_sandbox.sh tests 9-10 (MCP stdio/HTTP)

☑ Yes ☒ No

Pydantic Models Compliance

Verify strict implementation of:

- MBPPTaskInput
- SWEBenchTaskInput
- SolutionOutput
- StepMetrics
- SandboxConfig

Check field presence, correct types, JSON serialization,
and ISO timestamps.

StepMetrics must include:

- step, input_tokens, output_tokens, request_time_ms, timestamp
- api_url, model_name (per-step LLM metadata)
- llm_output (raw text generated by the LLM before code extraction)
- sandbox_input (Python code sent to the sandbox for execution)
- sandbox_output (sandbox execution result: stdout/stderr/error)
- retries (number of LLM API retries, 0 = first attempt succeeded)

SolutionOutput must include:

- task_id, benchmark, success, solution
- iterations, total_requests, total_input_tokens, total_output_tokens
- total_time_seconds, steps, error, timestamp
- system_prompt (the full system prompt sent to the LLM)

Yes No

Sandbox Security

Run the sandbox exam script:
./exams/exam_sandbox.sh --student-path ./student --moulinette-path ./moulinette --env-file .env

Verify import allowlist (tests 1-2):
Must block: os, subprocess, sys, socket, shutil, importlib,
builtins, serialization modules, dynamic imports (**import**)
Must allow: math, json, re, collections, itertools, typing
Sub-imports must also be blocked.

Verify file access restrictions (test 3):
Must block: /etc/passwd, path traversal (../../../etc/passwd),
/home, /root
Must allow: access inside allowed_directories only

Verify restricted builtins (test 4):
Must block: eval(), exec(), compile()
Safe builtins (print, len, range, type) must still work.

Verify no network access (test 5):
Must block: socket, urllib, http.client, requests
Non-network imports (json, math) must still work.

The sandbox security must be implemented using only standard
Python libraries and builtins. External sandbox libraries
(e.g., RestrictedPython) are not allowed.

Live sandbox verification:

- Open the student's sandbox interactively (uv run sandbox)
- Verify it can execute print("hello world")
- Verify final_answer("test") is callable
- Try a blocked import (e.g., import os) and verify it fails
- Try accessing a blocked path and verify it fails

Reference: exams/exam_sandbox.sh tests 1-5

Yes No

MCP Integration & Tool Discovery

Verify MCP server support (exam_sandbox.sh tests 6-10, 12-13):

Transport support:

- stdio transport works (--mcp-stdio)
- HTTP transport works (--mcp-server)

Tool servers:

- mcp_tools_mbpp.py provides: run_tests
- mcp_tools_swebench.py provides all mandatory tools from section 4.5: read_file, edit_file, list_files, search_code, search_function_or_class_definition_in_code, find_references, run_tests, get_patch, run_command

MCP behavior:

- final_answer is a sandbox feature, not an MCP tool (see §4.2)
- Dynamic tool discovery: connect the simple_mcp_server.py and verify the sandbox automatically discovers and exposes add, multiply, echo
- Sandbox manual generation: the sandbox generates tool documentation from MCP tool schemas (tool names, descriptions, parameter types)
- Manual updates when a different MCP server is connected

Reference: exams/exam_sandbox.sh tests 6-10, 12-14

No public or third-party MCP servers are allowed. Students must only use their own MCP server implementation. If any external MCP server is connected, fail this question.

☑ Yes ☒ No

Sandbox Features & Feedback

Verify sandbox resource enforcement:

- Timeout: code exceeding max_execution_time_seconds is interrupted
- Memory: code exceeding max_memory_mb is interrupted

Verify sandbox feedback transparency:
The sandbox must provide explicit feedback to the LLM in these cases:

- No valid code block found in the model's response
- A code block was malformed but was interpreted anyway
- Code hit the timeout (partial output should be preserved)
- Tool output was truncated due to size limits

The LLM should never be left guessing about what happened.

Reference: exams/exam_sandbox.sh tests 10-11, 14-15

☑ Yes ☒ No

MBPP Agent Results

By now (~5 min after Q2), the MBPP exam should be complete.
Check the terminal running exam_mbpp.sh.

Verify:

- Max iterations <= 10
- Max input tokens <= 6,000
- Max output tokens <= 1,500
- Timeout <= 120 seconds
- Pass threshold: 4/5 tasks
- No crash

If MBPP is still running, continue with Q9-Q11 and return to check results.

Reference: exams/exam_mbpp.sh

Yes No

MBPP Agent Excellence

Rate the quality of the MBPP agent beyond the minimum pass threshold.
Use solution.json files from the MBPP exam completed in Q8.

If Q8 did not pass, give 1 star.

1 star - Minimum pass only:
Passed 4/5 with high iteration counts (close to 10) and high tokens

2 stars - Solid:
Passed 4/5 with moderate efficiency (avg iterations < 7, tokens well under limits)

3 stars - Good:
Passed 5/5, avg iterations < 5, tokens under 50% of limits

4 stars - Excellent:
Passed 5/5, avg iterations < 3, tokens under 30% of limits

5 stars - Outstanding:
Passed 5/5, most tasks solved in 1-2 iterations, minimal token usage

Inspect with: cd moulinette && uv run moulinette_eval display ../path/to/solution.json

Rate it from 0 (failed) through 5 (excellent)

Metrics Tracking & Limits

Use a solution.json from the MBPP exam that just completed.

Verify:

- StepMetrics populated correctly for each step
- Accurate token tracking (input_tokens, output_tokens)
- request_time_ms tracked per step
- total_time_seconds correct
- api_url, model_name populated per step
- Agent respects iteration limit; token/time limits validated by moulinette

Check a solution.json file and verify:

- steps array has entries for each iteration
- Token counts are reasonable (not zeros or obviously fabricated)
- total_input_tokens equals sum of per-step input_tokens
- total_output_tokens equals sum of per-step output_tokens

You can use the moulinette display command for quick inspection:
cd moulinette && uv run moulinette_eval display ../path/to/solution.json

Yes No

Execution Trace Spot-Check

This test verifies that solution.json fields are wired to real
execution, not fabricated. Use an MBPP task.

Ask the student to make 4 live modifications, re-running the agent
after each one. Verify the change appears in solution.json.

Modification 1 — System prompt marker:
Add "[CORRECTION MARKER 42]" at the start of their system prompt.
Verify: system_prompt in solution.json starts with the marker.

Proves: system_prompt comes from the real prompt.

Modification 2 — Sandbox output marker:
Add print('SANDBOX_MARKER_42') before user code in sandbox execution.
Verify: sandbox_output contains "SANDBOX_MARKER_42" in every step.
Proves: sandbox_output comes from real execution.

Modification 3 — Sandbox code marker:
Prepend "# CORRECTOR_CHECK" to the code string before the sandbox receives it.
Verify: sandbox_input starts with "# CORRECTOR_CHECK" in every step.
Proves: sandbox_input comes from the real code path.

Modification 4 — Model name override:
Override the logged model_name to "correction-model-42" (without changing the actual API call).
Verify: model_name = "correction-model-42" AND the task still solves.
Proves: per-step metadata is wired independently of the API call.

Time: ~15-20 min total. Each modification takes 2-5 min.
If the student can't find where to make a change, they don't understand their own codebase — fail.

Student reverts all changes after (git checkout).

☑ Yes ☒ No

Anti-Cheating: Solution Provenance

This test verifies the student's agent solves tasks through legitimate code exploration and reasoning, not by looking up solutions.

=== Part A: Automated Checks ===

Run the anti-cheat exam script:
./exams/exam_anticheat.sh --student-path ./student

Review any warnings with the student.

=== Part B: SolutionOutput Inspection ===

Use the moulinette display command on a SWE-bench solution.json:
cd moulinette && uv run moulinette_eval display ../path/to/solution.json

Red flags in system_prompt:

- Contains task-specific solutions or patches
- References specific bug fixes or PRs
- Includes instructions to look up external resources

Red flags in sandbox_input:

- curl/wget to GitHub PR or issue URLs
- git log --all or git show accessing fix commits
- Fetching content from external URLs

=== Part C: AI-Assisted Cheat Detection ===

Copy the following prompt, the contents of the corresponding task.json AND solution.json into an LLM to analyze the agent's behavior.
Including task.json provides context so the LLM can distinguish legitimate file references from suspicious patterns:

Given this SWE-bench agent's system prompt and tool call history (from solution.json), analyze whether the agent solved the task through legitimate code exploration and reasoning, or whether there are signs of solution lookup. Specifically check for:

1. Does the agent follow a natural debugging workflow? (read code -> understand problem -> form hypothesis -> edit -> test)
2. Are there any tool calls that access external resources (GitHub PRs, issues, commits beyond the working

branch)?
3. Does the system prompt contain task-specific hints or solutions?
4. Is there evidence of the agent looking up known patches or fixes?

Flag any suspicious patterns and rate confidence: LOW / MEDIUM / HIGH.

=== Part D: Live Demonstration ===

Ask the student to explain their agent's solving strategy for
one of the SWE-bench tasks that passed. The student should be able to:

- Describe the general approach (how the agent explores, edits, tests)
- Explain the system prompt structure
- Walk through the tool call sequence in the solution.json
- Explain any non-obvious design choices

Yes No

Benchmark Report

Verify the student has produced a BENCHMARK_REPORT.md at the root
of their repository comparing models on SWE-bench tasks.

=== Part A: Structure Check (30s) ===

The report must contain ALL of:

1. Setup: 5+ models, 3+ tasks, selection rationale
2. Results table: pass/fail, iterations, input/output tokens, wall time
3. Provider reliability: avg response time, retries, availability
4. Intermediary metrics (at least 2 of):
    - Step at which agent first reads/edits the file in the final patch
    - Step at which test failures first decrease vs baseline
    - Iterations between "tests first pass" and "final_answer"
5. Ablation: before/after comparison of an agent change
6. Conclusions: model selection rationale based on data

Fail if: report missing, <5 models, <3 tasks, no token counts,
no provider reliability, no intermediary metrics, no ablation.

=== Part B: Backing Data Spot-Check (2 min) ===

Pick one solution.json that backs a report entry. Verify:

- sandbox_input / sandbox_output are populated
- Token counts in solution.json match the report
- The task_id matches

=== Part C: Student Explanation (2 min) ===

Ask the student to:

- Explain one insight from intermediary metrics
- Describe what the ablation changed and what was learned
- Justify why specific models were chosen or discarded

Fail if student can't explain their own report.

Yes No

Benchmark Report Depth

Rate the depth and quality of the benchmark report beyond minimum requirements.

If Q12 did not pass, give 1 star.

1 star - Bare minimum:
Exactly 5 models, 3 tasks, 2 intermediary metrics, 1 ablation, superficial conclusions

2 stars - Adequate:
5-6 models, 3-4 tasks, 2 metrics with interpretation, reasonable ablation

3 stars - Good:
7+ models or 5+ tasks, 3+ metrics, 2+ ablations, insightful conclusions

4 stars - Excellent:
8+ models, 5+ tasks, all metrics, 3+ ablations, statistical rigor

5 stars - Publication quality:
10+ models, 7+ tasks, visualizations, systematic ablation, cost analysis, reproducibility details, novel insights

Rate it from 0 (failed) through 5 (excellent)

Docker Lifecycle & Cleanup

By now, SWE-bench should be running or recently completed.
Use this to verify Docker cleanup.

Verify Docker management:

- Image pulled from docker_image field
- Container started correctly
- Container stopped and removed on normal exit
- Container cleaned on SIGINT / SIGTERM
- No orphan containers remain

Force kill the agent process during a SWE-bench task:

1. Start a SWE-bench task
2. Wait until the Docker container is running
3. Kill the agent process (kill -9 or Ctrl+C)
4. Verify: no orphan Docker containers remain (docker ps | grep sweb.eval should return nothing)

Reference: exams/exam_swebench.sh (container cleanup checks)

Yes No

SWE-bench Agent Results

By now (~45 min after Q2), the SWE-bench exam should be complete.
Check the terminal running exam_swebench.sh.

The script randomly selected 3 tasks from a pool of 6 predetermined tasks (all verified solvable by reference models). The student cannot predict which 3 tasks were selected.

Verify:

- Docker image pulled
- Container started
- eval_script executed
- Max iterations <= 30
- Max input tokens <= 300,000
- Max output tokens <= 10,000
- Timeout <= 900 seconds
- Pass threshold: 2/3 tasks
- Container cleanup on exit

Verify that any additional Docker dependencies installed by the student are reasonable (e.g., ruff, jedi) and do not circumvent the task.

If SWE-bench is still running, wait for it to complete before grading this question.

Reference: exams/exam_swebench.sh

☑ Yes ☒ No

SWE-bench Agent Excellence

Rate the quality of the SWE-bench agent beyond the minimum pass threshold.
Use solution.json files from the SWE-bench exam completed in Q14.

If Q14 did not pass, give 1 star.

1 star - Minimum pass only:
Passed 2/3, high iterations (close to 30), high tokens (close to 300k)

2 stars - Solid:
Passed 2/3 with moderate efficiency (avg iterations < 20)

3 stars - Good:
Passed 3/3, avg iterations < 15, tokens under 50% of limits

4 stars - Excellent:
Passed 3/3, avg iterations < 10, tokens under 30%, efficient exploration

5 stars - Outstanding:
Passed 3/3, avg iterations < 7, minimal tokens, fast convergence,
low gap between "tests pass" and "final_answer"

Inspect with: cd moulinette && uv run moulinette_eval display ../path/to/solution.json

Rate it from 0 (failed) through 5 (excellent)

Ratings

Don't forget to check the flag corresponding to the defense

✔ Ok ★ Outstanding project

Empty work | Incomplete work | Invalid compilation | Cheat | Crash | Incomplete group

▲ Concerning situation | ⊘ Forbidden function | Can't support / explain code

Conclusion

Leave a comment on this evaluation (2048 chars max)

Finish evaluation